

Practical 8

Part A — malloc/calloc/realloc/free

```
gcc -Wall -Wextra -g memory_alloc.c -o memory_alloc
./memory_alloc
valgrind --leak-check=full ./memory_alloc
```

Expected Valgrind result: all allocated blocks should be freed.

Part B — Copy-on-Write

```
gcc -Wall -Wextra -g cow_demo.c -o cow_demo
./cow_demo
```

fork() creates a child with logically copied address space. Linux can initially share physical pages and mark them copy-on-write. When the child writes to a shared page, a private copy is created.

Cow_demo.c -

```
#define _GNU_SOURCE
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define N (1024 * 1024)
```

```
static void print_rss(const char *label) {
```

```
    FILE *f = fopen("/proc/self/status", "r");
```

```
    if (!f) return;
```

```
    char line[256];
```

```
    while (fgets(line, sizeof(line), f)) {
```

```

    if (strncmp(line, "VmRSS:", 6) == 0) {
        printf("%s: %s", label, line);
        break;
    }
}
fclose(f);
}

```

```

int main(void) {
    char *data = malloc(N);
    if (!data) { perror("malloc"); return 1; }
    memset(data, 'A', N);

    printf("Parent PID: %d\n", getpid());
    print_rss("Parent before fork");

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); free(data); return 1; }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        print_rss("Child immediately after fork");
        printf("Child modifies one page...\n");
        data[0] = 'C';
        print_rss("Child after modification");
        printf("Child data[0] = %c\n", data[0]);
        _exit(0);
    }
}

```

```

waitpid(pid, NULL, 0);

printf("Parent data[0] = %c\n", data[0]);

print_rss("Parent after child exits");

free(data);

puts("COW idea: after fork(), pages are initially shared; a write causes a private copy
of the modified page.");

return 0;
}

```

Memory_alloc.c-

```

#include <stdio.h> #include <stdlib.h> #include <string.h>

int main(void) { printf("malloc/calloc/realloc/free demonstration\n");

int *a = malloc(5 * sizeof(*a));
if (!a) { perror("malloc"); return 1; }
for (int i = 0; i < 5; i++) a[i] = (i + 1) * 10;
printf("malloc: ");
for (int i = 0; i < 5; i++) printf("%d ", a[i]);
puts("");

int *b = calloc(5, sizeof(*b));
if (!b) { perror("calloc"); free(a); return 1; }
printf("calloc initially: ");
for (int i = 0; i < 5; i++) printf("%d ", b[i]);
puts("");

int *tmp = realloc(a, 10 * sizeof(*a));
if (!tmp) { perror("realloc"); free(a); free(b); return 1; }
a = tmp;
for (int i = 5; i < 10; i++) a[i] = (i + 1) * 10;
printf("realloc: ");
for (int i = 0; i < 10; i++) printf("%d ", a[i]);
puts("");

free(a);
free(b);
puts("free: all dynamically allocated blocks released.");
}

```

```
return 0;
```

```
}
```